

The Application Layer: More on HTTP

Lecture 6



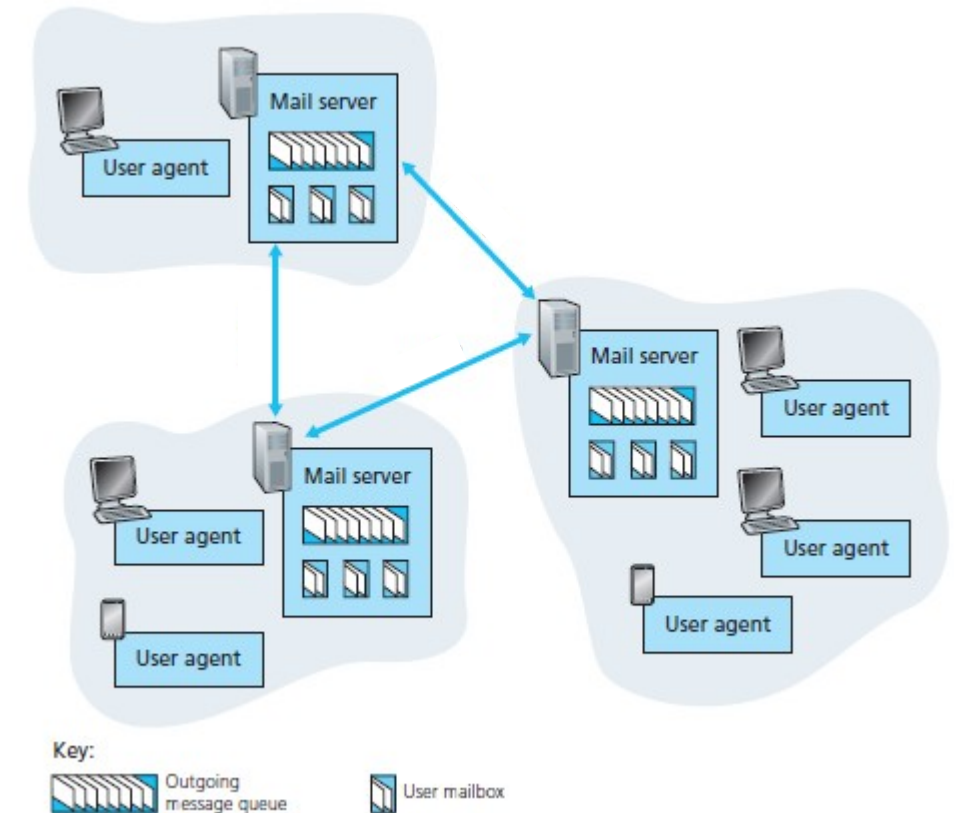
UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



Application Layer

Mails and SMTP

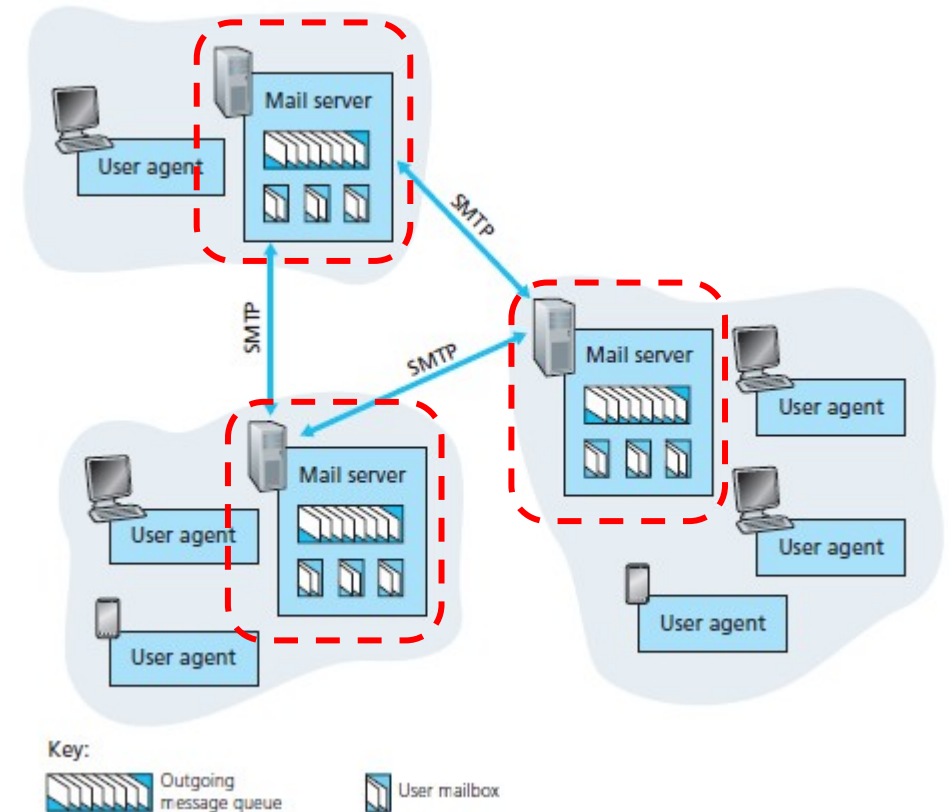
- Electronic mailing (e-mail) is one of the oldest and most important internet applications.
- Mailing is typically implemented in a client-server fashion, having 2 types of hosts:
 - **User agent**: application allowing mail management (e.g., Pine, K-9 Mail etc.).
 - **Mail Server**: a server that stores mails and maintain user-specific mailboxes.
- Differently from HTTP, here **communication also occurs between servers**. We need 2 applications/protocols:
 - One **between user agent and mail server**.
 - One **between mail server and mail server**.



Application Layer

Mails and SMTP

- Users **do not exchange mails directly** (as in instant messaging). It is preferred to use the more reliable and specialized mail servers.
- The **Simple Mail Transfer Protocol** (SMTP) is the main application-layer protocol working between **mail servers**.
 - It allows servers to exchange mails, **user agents do not use SMTP**.
 - Once a new mail arrives on the server, it is stored inside the **user-specific mailboxes**, waiting to be download locally by the user agent.
 - In SMTP there are a **client-side** (sender) and a **server-side** (receiver) that run on mail servers, both using the reliable **TCP** to transfer mail.

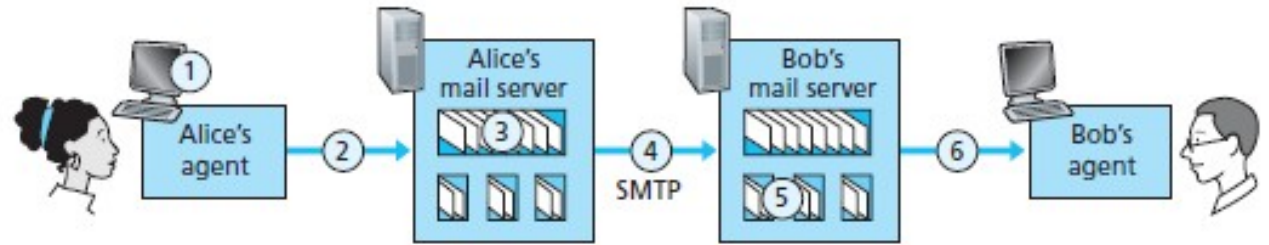


Application Layer

SMTP

- Let's assume to have host A (Alice) that is sending an e-mail to host B (Bob), we have 4 elements involved:

- Alice's agent.
- Alice's mail server.
- Bob's agent.
- Bob's mail server.



- The process works as follows:

1. Alice **invokes her user agent** for e-mail, provides Bob's e-mail address (e.g., bob@someschool.edu), composes a message, and instructs the user agent to send the message.
2. Alice's **user agent sends the message to her mail server**, where it is placed in a message queue.
3. The client side of SMTP, running on Alice's mail server, sees the message in the message queue. It **opens a TCP connection to an SMTP server**, running on Bob's mail server.
4. After some initial SMTP handshaking, the SMTP client sends Alice's message **into the TCP connection**.
5. At Bob's mail server, **the server side of SMTP receives the message**. Bob's mail server then places the message in **Bob's mailbox**.
6. Bob **invokes his user agent** to read the message at his convenience.

Application Layer

SMTP

- First, the client SMTP (running on the sending mail server host) **establishes a TCP a connection to port 25 at the server SMTP** (running on the receiving mail server host).
- If the server is down, the client tries again later. Once this connection is established, **the server and client perform some application-layer handshaking:** SMTP clients and servers introduce themselves before transferring information.
 - During this SMTP handshaking phase, the SMTP client indicates the e-mail address of the sender (the person who generated the message) and the e-mail address of the receiver.
- SMTP can count on **the reliable data transfer service of TCP** to get the message to the server without errors.

Application Layer

Accessing mails

- Deployment of SMTP servers is **more reliable** with respect to direct user-to-user mailing:
 - Servers are, by definition, **always on** and visible to all devices of the network.
 - Servers are certified.
 - The server may continuously try to re-send mails if delivery fails (which is a quite computationally expansive process).
- On the other hand, to access the mails from the server **an additional client-server application is needed** (and a different protocol), most popular are:
 - Post Office Protocol—Version 3 (**POP3**).
 - Internet Mail Access Protocol (**IMAP**).
 - **HTTP**.

Application Layer

Accessing mails: POP3

- The **Post Office Protocol - Version 3** (POP3) is an extremely simple mail access protocol.
- The user agent (the client) opens a **TCP connection** to the mail server (the server) on port 110.
- With the TCP connection established, POP3 progresses through three phases:
 1. **Authorization**: the user agent **sends a username and a password** to authenticate the user.
 2. **Transaction**: the user agent can **retrieve messages**, mark messages for deletion, remove deletion marks, and obtain mail statistics.
 3. **Update**: after the client has issued the quit command, ending the POP3 session, the mail server **deletes the messages** that were marked for deletion.

Application Layer

Accessing mails: IMAP

- With POP3 access, messages can just be **downloaded** or **deleted**. Action like searching or organizing mails into folders are not considered (these must be done on the local machine).
- The **Internet Mail Access Protocol** (IMAP) allow servers to provide **additional features**:
 - Managing and **creating folders**.
 - Perform **search** into remote folders for messages matching specific criteria.
 - Allow user agent to obtain just parts of messages (e.g., header only, attachments only, etc.).
 - Allow **multiple clients** to be connected to the same server.

Application Layer

Accessing mails: HTTP

- More and more users today are sending and accessing their e-mails through **their Web browsers**.
- HTTP Web-based access was introduced by Hotmail in the mid 1990 and is now the mainstream approach (Google, Yahoo!, Virgilio, etc.) and it is used by almost every major university and corporation (UniNA included).
- With this service, **the user agent is an ordinary Web browser**, and the user communicates with its remote mailbox via HTTP rather than through the POP3 or IMAP protocols.
- This works for user-agents, while **mail servers still rely on the standard SMTP** to exchange messages.

Application Layer

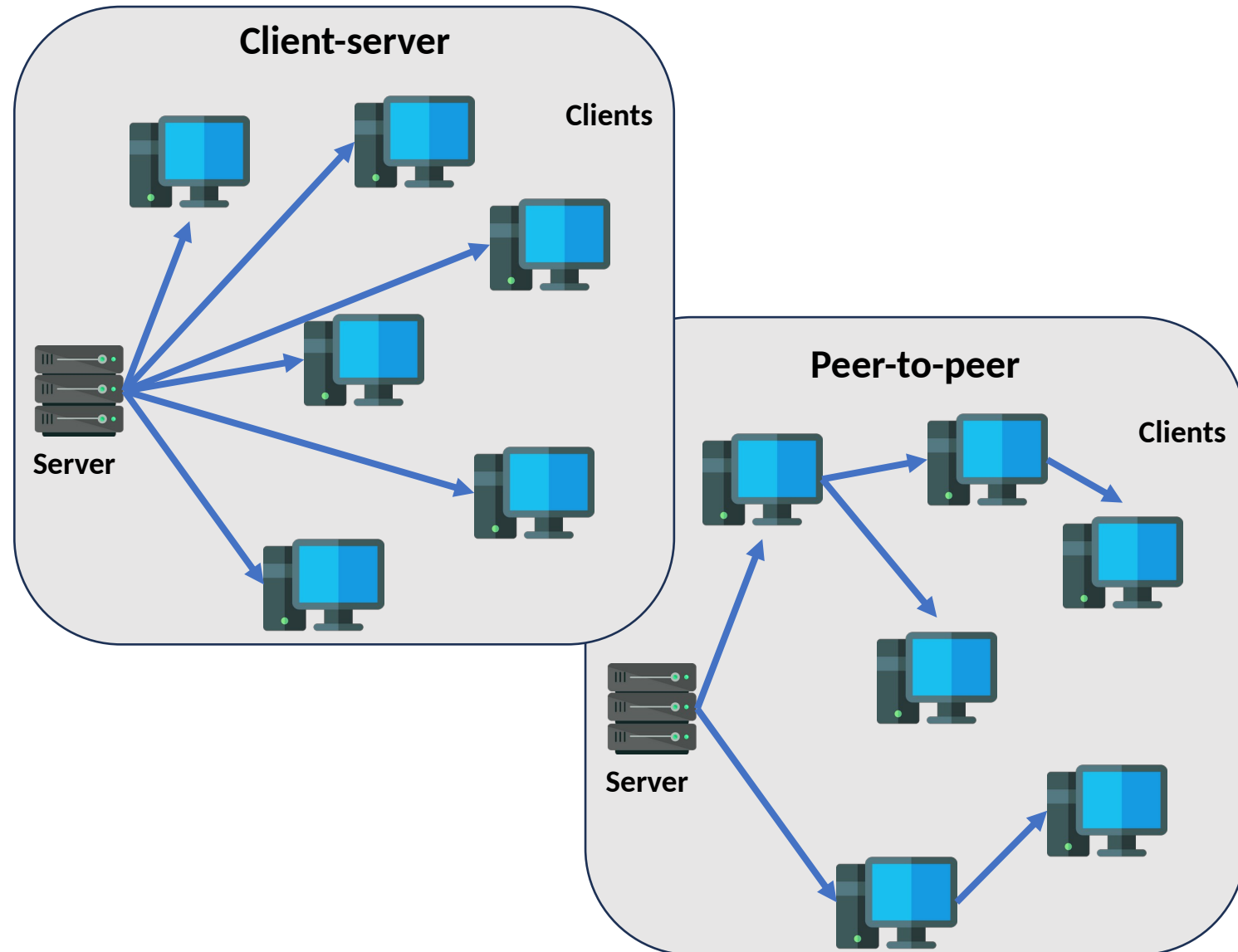
Peer-to-Peer

- Differently from client-server, P2P architecture **makes minimal (if none) use of servers**, here we have **pairs of intermittently connected hosts** (peers) that communicate directly with each other.
- The peers are **not owned by a service provider** but are instead desktops and laptops controlled by users.
- One natural application of P2P is **file sharing**:
 - In a **client-server** architecture, the server must **share files to all clients** (which is a serious burden on the server and requires large bandwidth).
 - In a **P2P** architecture, the peers that receive the file may also **share it to other peers**. Somehow, peers are clients and servers at the same time.

Application Layer

Peer-to-Peer

- In file sharing, the P2P approach typically **scales better** than client-server one.
- On the other hand, P2P approaches can be quite **complex to implement**.
- There may also be **security issues** in having all this clients in direct communication.



Application Layer

BitTorrent

- A popular P2P protocol for file sharing/distribution is **BitTorrent**.
 - BitTorrent estimated users are 150-170 million (in 2023).
- A **torrent** is the collection of all peers participating in the distribution of a particular file. Peers in a torrent download **equal-size chunks** of the file from one another (typical chunk size is 256 kbytes).
- When a peer joins a torrent (having no chunks):
 - It starts by **accumulating chunks**.
 - While the peer downloads chunks **it also uploads chunks** to other peers.
 - Once the peer **acquires the entire file**, it may (selfishly) **leave the torrent**, or (altruistically) **stay in the torrent** and continue to upload chunks to other peers.
- Peers may **leave the torrent at any time** with only a subset of chunks, and later rejoin the torrent.

Application Layer

BitTorrent: tracker

- Each torrent has an infrastructure node called **tracker** that takes track of the peers participating to the torrent (trackers are basically **servers**).
- When a **new peer** joins a torrent:
 - It **registers** itself with the tracker and **periodically informs** the tracker about its status.
 - The tracker **provides the IP addresses** of a randomly selected subset of peers from the torrent.
- Having the list of peers, the new host attempts to establish concurrent TCP **connections with all the peers on this list** (neighbors).
 - During the execution, some of the connected peers **may leave** while other peers (outside the initial list) may attempt to **establish new connections**.
- At any given time, each peer will have a **subset of chunks from the file**, with different peers having different subsets. Periodically, a host will ask each connected peers (over the TCP connections) for **the list of the chunks they have**.

Application Layer

BitTorrent: client

- The **downloading client** decides which chunks to request (and to whom) following a rarest-first principle:
 - **Rarest-first**: the chunks with **fewest repeated copies** among the neighbors are prioritized. In this way, the **rarest chunks get more quickly redistributed**, so to (roughly) equalize the numbers of copies in the torrent.
- The **uploading client** decides which requests are served following two intertwined principles:
 - **Trading**: hosts give priority to **the best 4 neighbors** that are currently supplying data at the **highest rate**. This check is periodically performed (10 seconds) and the list of 4-best neighbors is updated.
 - **Random selection**: every 30 seconds, a host also picks **one additional neighbor** at random and sends it chunks. If this **random exchange is good** (the 2 hosts are good partners) **they will enter the respective best lists**. This process also allows peers with compatible upload rates to find each other.